Research and Planning
Django eCommerce Application — Part 1

Course: Introduction to Software Engineering **Task:** 18

Student: Tony Young **Prepared:** November 2026

Repository: https://github.com/tonyyoung2611/django-ecommerce-part1

1. Project purpose and scope

Develop a server-rendered Django eCommerce application that supports two user roles: buyers and vendors. Buyers browse products, maintain a session-based cart, check out, and submit reviews. Vendors manage their own stores and product listings. This document records the design and implementation approach for Part 1, including the database choice and key implementation considerations.

2. Requirements and user workflows

• **Authentication:** Register an account, select buyer or vendor role, log in and out, and use password-reset views.

• **Buyer journey:** Browse products, add/remove items from the cart, check out, view order confirmation, and leave a product review.

• **Vendor journey:** Access a vendor dashboard; create, edit, and delete owned stores and products.

• **Order integrity:** Validate available stock at checkout, record order line items, and reduce stock when an order is placed.

• **Administration:** Register the six domain models in Django admin.

3. Technology research and selection

Technology	Purpose / rationale
Python + Django	Server-side views, models, forms, authentication, templates and URL routing.
MariaDB	Relational persistence for linked users, stores, products, orders and reviews; configured through Django MySQL backend.
mysqlclient	Database driver used by Django to connect to MariaDB.
HTML templates	Render product, cart, authentication and vendor pages.
Git + GitHub	Track changes, publish source and support reproducible setup.

4. Data model and relationships

• **Store:** name, description, owner (user). One vendor can own multiple stores.

• **Product:** store, name, description, price, stock. One store has many products.

• **Profile:** one-to-one user relationship, with buyer or vendor role.

• **Order:** buyer, creation timestamp, total. A buyer can place many orders.

• **OrderItem:** order, product, quantity, captured price. An order contains multiple items.

• **Review:** product, user, rating, comment, verified-purchase flag and timestamp.

5. Architecture and request flow

Browser request → Django URL route → view → model/form validation → MariaDB query or update → HTML template/redirect. The cart is held in the Django session until checkout. During checkout, a database transaction locks selected products, checks stock, creates the order and order items, and updates inventory. An invoice message is generated through Django's email system.

6. Database and migration plan

Use MariaDB rather than the initial SQLite configuration. Create the ecommerce_db database and a dedicated ecommerce_user account with appropriate permissions. Configure Django DATABASES with ENGINE django.db.backends.mysql, localhost:3306 and utf8mb4; load the password from ECOMMERCE_DB_PASSWORD rather than committing it to source control. Use Django migrations to create or update the relational schema. Verify with manage.py showmigrations, makemigrations --check --dry-run, and check. The configured database connection and migration status were checked during the Part 1 corrections; a full fresh-install migration test remains advisable before final assessment.

7. Security and validation considerations

• Require login for checkout, review submission and vendor operations; verify the correct buyer/vendor role.

• Filter store and product edits by the authenticated owner to prevent cross-vendor changes.

• Use POST for cart modifications and deletion; use Django CSRF protection in forms.

• Use transaction.atomic and select_for_update to reduce stock-race risks.

• Protect products referenced by order items using PROTECT and handle ProtectedError on deletion.

• Store Django SECRET_KEY and database credentials outside Git; exclude local secrets and virtual environments with .gitignore.

8. Implementation sequence

• Create Django project and shop application; configure settings and routes.

• Define six models and create/apply migrations in MariaDB.

• Build registration and role-specific forms and views.

• Implement product browsing, session cart, checkout and order confirmation.

• Implement reviews and vendor store/product management.

• Add HTML templates, admin registrations, documentation and docstrings.

• Run Django checks; publish source to GitHub with README and requirements.txt.

9. Testing and review plan

Run python manage.py check and python manage.py makemigrations --check --dry-run. Manually test buyer registration, login, adding/removing cart items, checkout with sufficient and insufficient stock, order confirmation access restrictions, review verification, vendor ownership restrictions, and protected product/store deletion. Test password-reset and invoice output using the console email backend. Automated tests are a recommended follow-up; the existing tests.py currently contains no custom tests.

10. Known limitations and future improvements

Part 1 uses server-rendered pages and session carts; it does not implement the Part 2 REST API. The console email backend is for development and does not deliver real emails. Additional automated tests, robust validation, deployment settings, and a clean-database installation test would improve reliability. This planning record describes the implemented Part 1 scope and planned verification; it does not claim all manual tests have been completed.

References

Django documentation: https://docs.djangoproject.com/en/6.1/ | MariaDB: https://mariadb.org/ | PEP 257: https://peps.python.org/pep-0257/